

Deepfake Detection Application

Renee Ngari, Sushmita Basnet

Spring / Summer / Fall 2026

1 Research Context and Problem Statement

In recent times, progress in artificial intelligence and deep learning has led to the swift evolution of deepfake technologies, capable of creating extremely lifelike synthetic media such as images, videos, and audio. Although these technologies offer helpful uses in areas like entertainment, education, and healthcare, they also present significant cybersecurity risks. Deepfakes may be utilized for identity theft, spreading false information, biometric deception, and social engineering assaults, greatly eroding confidence in digital material.

The increasing availability of deepfake tools and datasets has intensified this problem. Publicly accessible datasets and open-source tools enable even amateurs to produce realistic synthetic media, heightening the potential for abuse. As emphasized in the paper, this extensive accessibility has turned deepfake creation into a worldwide cybersecurity issue, especially in domains like digital identity confirmation and online authentication mechanisms.

To combat these threats, scientists have created numerous methods for detecting deepfakes. These consist of:

- CNN-based frameworks for identifying spatial discrepancies in images
- RNN and LSTM architectures for detecting time-based anomalies in videos
- Models based on transformers for identifying intricate patterns within the data.
- Multi modal systems integrating auditory and visual analysis.

Although these methods have demonstrated encouraging results, they remain constrained in practical situations. A significant problem is inadequate generalization across datasets, where models developed on a single dataset struggle on others because of variations in resolution, lighting, and manipulation methods. Moreover, detection systems are susceptible to adversarial assaults, where minor alterations can trick AI models into incorrect classifications

Privacy and ethical issues continue to pose significant challenges. Numerous detection techniques depend on biometric information like facial recognition or voice analysis, leading to worries about data abuse and user privacy. Thus, it is crucial to balance security with ethical data management to create reliable systems

2 Proposed Solution

The threats outlined above deepfake-enabled identity theft, biometric deception, and the erosion of trust in digital media demand a detection system that is not only accurate but also practical for everyday users. Existing research demonstrates that deep learning models can identify subtle artifacts left behind by generative processes; however, these models are rarely packaged in a form that is lightweight, privacy-respecting, and accessible outside of a laboratory setting. To close this gap, we propose **DeepGuard**, a modular, real-time deepfake detection framework designed for seamless integration into existing digital platforms such as social media feeds, video conferencing tools, and web browsers.

2.1 System Architecture

DeepGuard will be organized around three tightly coupled components: a *pre-processing pipeline*, a *detection engine*, and a *user-facing interface layer*. Together, these components address the core failure modes identified in prior work—poor cross-dataset generalization, vulnerability to adversarial perturbations, and the privacy risks associated with storing biometric data on remote servers.

2.1.1 Preprocessing Pipeline

Because deepfake detectors that operate on raw pixels tend to overfit to the resolution and lighting conditions of their training data, DeepGuard’s preprocessing pipeline will normalize inputs before they reach the detection engine. For video content, frames will be extracted at a fixed rate (target: 5 frames per second) to balance temporal coverage against computational cost. Each frame will undergo:

- **Face alignment and cropping** using a lightweight landmark detector (e.g., MediaPipe Face Mesh), so that the detection model always receives a consistently framed facial region regardless of camera angle or image resolution.
- **Frequency-domain transformation** via a Fast Fourier Transform (FFT), which converts spatial pixel information into frequency components. Prior work has shown that GAN-generated faces exhibit characteristic high-frequency artifacts invisible to the naked eye but detectable in the frequency domain [?]. Providing both spatial and frequency representations as input channels directly addresses the generalization problem caused by surface-level pixel variation across datasets.
- **On-device processing:** all preprocessing will execute locally on the user’s device. No raw facial data will be transmitted to an external server, mitigating the biometric privacy risks described in the problem statement.

2.1.2 Detection Engine

The core of DeepGuard is an **Attention-Enhanced Lightweight Convolutional Neural Network (AE-LightCNN)** inspired by architectures such as MobileNetV3 and EfficientNet-Lite, which are designed for inference on resource-constrained hardware. The choice of a lightweight backbone directly solves the impracticality of computationally heavy transformer-based detectors, which cannot run in real time on consumer hardware.

The network will accept a dual channel input: the aligned facial crop (spatial domain) and its FFT magnitude spectrum (frequency domain). A channel attention module (Squeeze-and-Excitation block) will allow the model to dynamically weight the relative importance of spatial versus frequency cues depending on the manipulation type it encounters a design choice motivated by the observation that different generation methods leave stronger artifacts in different domains.

To improve cross data set generalization, one of the central problems identified in prior literature training will incorporate two strategies:

1. **Multi dataset training:** The model will be trained jointly on at least three publicly available benchmark datasets (FaceForensics++, Celeb-DF, and DFDC), exposing it to a wide range of manipulation methods, compression levels, and recording conditions. This approach is motivated by evidence that single-dataset training leads to brittle models that fail when tested on out-of-distribution examples.
2. **Augmentation-based robustness training:** During training, we will apply randomized video compression (H.264 at varying CRF values), JPEG recompression, Gaussian noise, and random blurring to simulate the degradation that deepfakes undergo when uploaded and re-encoded on social media platforms. This directly addresses the finding that real-world content is rarely as clean as content in research datasets.

To defend against adversarial attacks where a malicious actor applies small perturbations designed to fool the detector, the training procedure will include **adversarial training** using the Fast Gradient Sign Method (FGSM). By exposing the model to adversarial examples during training, we aim to produce a detector that is robust to the kinds of evasion strategies an attacker might employ.

2.1.3 User-Facing Interface Layer

Even a perfectly accurate detector is useless if it cannot reach ordinary users. DeepGuard will expose its detection capability through two delivery mechanisms that solve the accessibility problem identified in the problem statement:

- **Browser Extension:** A lightweight JavaScript wrapper will invoke an on-device WebAssembly (WASM) build of the detection engine. When a user encounters a video on a supported platform, the extension will

display a non intrusive confidence indicator (e.g., a colored badge: green for authentic, yellow for uncertain, red for likely synthetic) overlaid on the content. This design ensures that detection happens entirely in the browser without requiring data to leave the user’s machine.

- **REST API:** For platform-level integration (e.g., social media moderation pipelines), DeepGuard will also be deployable as a containerized microservice exposing a simple REST endpoint. Platforms can submit a video URL or binary payload and receive a structured JSON response containing a per frame authenticity score and an aggregate verdict. This allows for automated moderation at scale without requiring each platform to train or maintain its own model.

2.2 Addressing Privacy and Ethical Constraints

Because the problem statement identifies user privacy as a first class concern, DeepGuard is architected from the ground up to minimize data collection. The on device processing model ensures that facial biometrics are never stored or transmitted. In the REST API deployment, submitted media will be deleted immediately after scoring. No user-identifying metadata will be retained. These design choices are consistent with principles of *privacy by design* and are intended to ensure that the tool does not itself become a vector for surveillance or data abuse.

2.3 Summary of How the Proposed Solution Addresses the Problem

Table 1 maps each challenge identified in the problem statement to the specific component of DeepGuard designed to address it.

Table 1: Problem–Solution Alignment

Problem	DeepGuard Solution
Poor cross-dataset generalization	Multi-dataset training + FFT dual-channel input
Vulnerability to adversarial attacks	Adversarial training with FGSM
High computational cost prevents real-time use	Lightweight MobileNet backbone + WASM browser build
Privacy risks from biometric processing	On-device inference; no data stored or transmitted
Detection tools inaccessible to ordinary users	Browser extension with plain-language confidence indicator
Platform-level moderation at scale	Containerized REST API with per-frame scoring

By combining a frequency aware dual channel architecture, multi dataset generalization training, adversarial robustness, and privacy preserving on device inference, DeepGuard addresses not just the technical limitations of current

detectors but also the practical and ethical barriers that have prevented effective deepfake detection from reaching end users. The next section will describe how we will evaluate whether this system meets these goals and the timeline by which we plan to complete each phase of the work.

3 Evaluation and Implementation Plan

3.1 Evaluation Plan

To evaluate the effectiveness of this deepfake detection tool, we will focus on its accuracy and performance in real-world scenarios.

First, we will train and test the model using a dataset that includes both real and deepfake images. The data will be divided into training, validation, and testing sets, allowing me to measure performance on new, unseen content. We will use metrics such as accuracy, precision, recall, and F1-score for evaluation. While accuracy provides an overview of performance, precision and recall are particularly important in this context, as the goal is to accurately identify fake content without frequently misclassifying real content.

Next, we will simulate real-world usage of the tool by testing it on mixed sets of real and fake images, similar to what users encounter on social networks. Fortunately, this will help assess the model's performance in a more realistic environment. Additionally, We will monitor the speed at which the model processes images, as it is crucial for it to function in real-time or close to it.

We will conduct multiple experiments with various machine learning models and settings to determine which approach is most effective. For instance, we may compare different convolutional neural network (CNN) models or adjust parameters such as learning rate and the number of layers. Each version will be evaluated using the same metrics to facilitate a clear comparison of results.

Furthermore, we will analyze instances where the model makes errors. By examining misclassified images, we can identify any specific types of deepfakes that are more challenging to detect, such as higher-quality or more realistic ones.

The solution will be deemed successful if the model achieves strong performance on the test data (around 80 percents or higher accuracy with balanced precision and recall), operates quickly enough for near-real-time use, and does not exhibit significant overfitting. If the model performs poorly on new data, operates too slowly, or frequently misclassifies content, improvements will be necessary.

To evaluate DeepGuard, we will measure performance using unseen testing data containing both authentic and manipulated images.

Metrics will include:

- Accuracy
- Precision
- Recall

- F1-score
- Confusion Matrix
- Inference Speed

The project will be considered successful if it achieves:

- Accuracy of at least 85%
- Precision of at least 85%
- Recall of at least 80%
- F1-score above 0.82
- Processing speed under 1 second per image
- Minimal overfitting between training and validation sets

We will also test the model using mixed real world content similar to media found on social platforms.

3.2 Timeline

Week 1: Project Setup + Direction

- **Finalize goal (real-time deepfake detection tool)**
- **Research 1–2 strong detection approaches (don't overdo it)**
- **Collect dataset (real + deepfake images)**
- **Set up coding environment**

Week 2: Data Preparation

- **Clean, label, and preprocess data (resize, normalize)**
- **Split into training, validation, and test sets**
- **Verify dataset quality and balance**

Week 3: Baseline Model (Move Fast)

- **Build and train a simple CNN model**
- **Record baseline performance (accuracy, precision, recall, F1-score)**
- **Identify major weaknesses**

Week 4: First Improvements

- **Apply fixes (data augmentation, dropout, basic tuning)**
- **Retrain and compare results to baseline**
- **Start tracking experiments clearly**

Week 5: Model Exploration (Limited but Focused)

- **Test 1–2 alternative models or architectures**
- **Compare all models using the same metrics**
- **Start checking processing speed (important for real-time goal)**

Week 6: Select Best Model + Refine

- **Choose the best-performing model so far**
- **Fine-tune hyperparameters (learning rate, batch size, etc.)**
- **Reduce overfitting and stabilize performance**

Week 7: Real-World Testing

- **Test model on mixed/unseen image sets (simulate real usage)**
- **Measure false positives and false negatives**
- **Evaluate speed (can it run close to real time?)**

Week 8: Error Analysis + Final Improvements

- **Analyze misclassified images**
- **Identify patterns in errors**
- **Make final targeted improvements**

Week 9: Final Evaluation + Results

- **Run full testing on final model**
- **Generate confusion matrix and performance metrics**
- **Create graphs and summarize results**

Week 10: Poster + Final Deliverables

- **Build poster (problem, method, results, impact)**
- **Explain how the tool would integrate into browsers/social media**
- **Practice presentation and finalize report**

4 Conclusion

Learning more about deepfakes detection permitted us to find an adequate solution that we are committed to developing. Using a highly effective deepfake detection tool that will seamlessly integrate with popular platforms such as social media and web browsers. Our focus is on delivering real-time content analysis, enabling users to swiftly identify any potential fakes. By utilizing advanced machine learning techniques, we will pinpoint the subtle details and patterns that distinguish deepfakes from authentic content, and at the same time, strengthen online trust. As we embark on this project, we will review existing detection methodologies and rigorously test various models to identify the most powerful solutions. Our ultimate aim is to create a practical and reliable resource that empowers users to make informed decisions about the authenticity of the content they encounter online.

5 Links

1. EPJ Conferences Paper:
<https://www.epj-conferences.org/articles/epjconf/pdf/2025/29/epjconfaiptec202501019.pdf>
2. ResearchGate Paper:
<https://www.researchgate.net/publication/390978624LightweightDeepfakeDetectiononMobileDevices>
3. ScienceDirect Article:
<https://www.sciencedirect.com/science/article/pii/S2215016125004765>

5.1 References

1. Amol Dhumane, Sonam Singh. *Unmasking Digital Deceptions: An Integrative Review of Deepfake Detection, Multimedia Forensics, and Cyber-security Challenges*.